

Calories Prediction — Machine Learning

English

This notebook covers model training, evaluation, and persistence, using the project's own scripts:

- `src/evaluate_model.py` — trains Linear Regression, Decision Tree, and Random Forest, and prints a full comparison table (MAE, MSE, RMSE, R^2) plus the best model by R^2 .
- `src/train_model.py` — trains the same three models and saves the best one (by R^2) to `models/best_model.pkl` with `joblib`.

Unlike `data_analysis.py` / `data_cleaning.py` / `preprocessing.py` / `visualization.py`, these two files are **not** built as reusable functions — they are standalone scripts that run their full logic at import time. So instead of calling functions, this notebook imports each module directly (`import src.evaluate_model`, `import src.train_model`), which runs the script exactly as it would from the command line — same prints, same training, same saved file — and afterward every top-level variable in each script (`results_df`, `best_model`, `X_test`, `y_test`, etc.) stays available as an attribute of the imported module for further inspection here, without rewriting any of their logic.

العربية

يغطي هذا الدفتر تدريب النموذج وتقييمه وحفظه، باستخدام ملفات المشروع نفسها:

- `src/evaluate_model.py` — يدرب Linear Regression وDecision Tree وRandom Forest، بالإضافة إلى أفضل نموذج حسب (MAE , MSE , $RMSE$, R^2) ويطبع جدول مقارنة كامل.
- `src/train_model.py` — يدرب نفس النماذج الثلاثة ويحفظ الأفضل إلى `models/best_model.pkl` باستخدام `joblib`.

بخلاف `data_analysis.py` / `data_cleaning.py` / `preprocessing.py` / `visualization.py`، هذان الملفان ليسا مبنيين كدوال قابلة لإعادة الاستخدام — بل هما ملفًا، سكربت كاملان ينفذان منطقهما بالكامل لحظة الاستيراد. لذلك بدلاً من استدعاء دوال، يقوم هذا الدفتر وهو (`import src.evaluate_model`, `import src.train_model`) باستيراد كل وحدة مباشرة ما يشغل السكربت تمامًا كما لو تم تشغيله من سطر الأوامر — نفس الطباعة، نفس التدريب، نفس (`results_df`, `best_model`) الملف المحفوظ — وبعدها يبقى كل متغير على مستوى الوحدة متاحًا كخاصية للوحدة المستوردة للفحص لاحقًا هنا، دون إعادة كتابة أي من (إلخ، `X_test`, `y_test`) منطقهما.

Project Setup

English

We add the project root to `sys.path` so `import src...` works from this notebook, the same way the scripts import from each other. Because `evaluate_model.py` and `train_model.py` run at import time, each is imported only once per kernel session — re-running these cells later requires restarting the kernel or using `importlib.reload()`.

العربية

من هذا الدفتر، بنفس الطريقة `import src...` حتى يعمل `sys.path` نضيف جذر المشروع إلى `evaluate_model.py` و `train_model.py` التي تستورد بها الملفات من بعضها البعض. ولأن إعادة تشغيل هذه — kernel ينفّذان لحظة الاستيراد، يتم استيراد كل منهما مرة واحدة فقط لكل جلسة `importlib.reload()` أو استخدام kernel الخلايا لاحقًا تتطلب إعادة تشغيل الـ.

```
In [1]: %matplotlib inline

import sys
from pathlib import Path

import joblib
import pandas as pd
import matplotlib.pyplot as plt

# -----
# Make src/ importable, regardless of where the notebook is launched from
# -----
NOTEBOOK_DIR = Path.cwd()
PROJECT_ROOT = NOTEBOOK_DIR.parent if NOTEBOOK_DIR.name == "notebooks" else NOTEBOOK_DIR

if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

print(f"Project root : {PROJECT_ROOT}")
```

Project root : c:\Users\ayman\OneDrive\Desktop\calories-prediction

Model Evaluation

English

Importing `src.evaluate_model` runs the full script: it loads the processed dataset, encodes `Gender`, splits into train/test, trains all three models, and prints the MAE / MSE / RMSE / R^2 comparison table plus the best model.

العربية

`Gender`، يشغّل السكريبت بالكامل: يحمّل البيانات المعالجة، يرمّز `src.evaluate_model` استيراد، يقسم البيانات إلى تدريب/اختبار، يدرب النماذج الثلاثة، ويطبع جدول مقارنة `MAE / MSE / RMSE /  $R^2$` .

بالإضافة إلى أفضل نموذج.

```
In [2]: import src.evaluate_model as evaluate_model
```

```
=====
                        MODEL EVALUATION
=====

-----
                        RESULTS
-----

      Model    MAE      MSE    RMSE     R2
Linear Regression 178.41  42871.96 207.06 -0.0185
Decision Tree    273.70 113156.10 336.39 -1.6883
Random Forest    167.92  38772.13 196.91  0.0789

=====
                        BEST MODEL
=====

Model : Random Forest
R²    : 0.0789

=====
                        METRIC MEANING
=====

MAE   : Lower is better
MSE   : Lower is better
RMSE  : Lower is better
R²    : Higher is better
```

Results Table

English: `evaluate_model.results_df` is the exact DataFrame built by the script; we just display it here for a nicer notebook rendering (the script already printed a formatted text version above).

العربي: `evaluate_model.results_df` هو نفس إطار البيانات الذي بناه السكريبت؛ نعرضه هنا فقط بشكل أوضح داخل الدفتر (السكريبت طبع أعلاه نسخة نصية منسقة بالفعل).

```
In [3]: evaluate_model.results_df.sort_values("R2", ascending=False).reset_index(drop=True)
```

```
Out[3]:
```

	Model	MAE	MSE	RMSE	R2
0	Random Forest	167.916500	38772.127790	196.906393	0.078886
1	Linear Regression	178.409025	42871.961923	207.055456	-0.018514
2	Decision Tree	273.700000	113156.100000	336.386831	-1.688263

Model Training & Persistence

English

Importing `src.train_model` runs its script: it loads the processed dataset again, encodes Gender , splits, trains the same three models, and saves the best one (by R^2) to `models/best_model.pkl` .

العربية

يقسم ، Gender يشغل سكريته: يحمل البيانات المعالجة من جديد، يرمز `src.train_model` استيراد `models/best_model.pkl` في (R^2 حسب) البيانات، يدرّب نفس النماذج الثلاثة، ويحفظ الأفضل .

```
In [4]: import src.train_model as train_model
```

```
Dataset loaded successfully  
Shape: (200, 10)
```

```
Training: Linear Regression  
MAE : 178.41  
MSE : 42871.96  
R2  : -0.02
```

```
Training: Decision Tree  
MAE : 273.7  
MSE : 113156.1  
R2  : -1.69
```

```
Training: Random Forest  
MAE : 167.92  
MSE : 38772.13  
R2  : 0.08
```

```
=====
BEST MODEL
=====
Model: Random Forest
R2: 0.08
```

```
Model saved to:  
C:\Users\ayman\OneDrive\Desktop\calories-prediction\models\best_model.pkl
```

```
In [5]: print(f"Best model      : {train_model.best_model_name}")  
        print(f"R2 (test set) : {round(train_model.best_r2, 4)}")  
        print(f"Saved to      : {train_model.BEST_MODEL_FILE}")  
        print(f"File exists    : {train_model.BEST_MODEL_FILE.exists()}")  
  
        # Sanity check: Load the saved file back and confirm it matches  
        reloaded_model = joblib.load(train_model.BEST_MODEL_FILE)  
        print(f"Reloaded model : {type(reloaded_model).__name__}")
```

```
Best model      : Random Forest
R2 (test set)   : 0.0789
Saved to        : C:\Users\ayman\OneDrive\Desktop\calories-prediction\models\best_model.pkl
File exists     : True
Reloaded model  : RandomForestRegressor
```

Actual vs Predicted (Best Model)

English

Using the exact `best_model`, `X_test`, and `y_test` that `train_model.py` already computed — the same model that was saved to disk — we plot predicted vs actual `Calories Burned` on the held-out test set. Points close to the diagonal line indicate accurate predictions.

العربية

بالفعل — `train_model.py` التي حسبها `y_test` و `X_test` و `best_model` باستخدام نفس `Calories Burned` وهو نفس النموذج المحفوظ على القرص — نرسم القيم المتوقعة مقابل الفعلية لـ `Calories Burned` على مجموعة الاختبار المحجوزة. النقاط القريبة من الخط القطري تعني توقعات دقيقة.

```
In [6]: predictions = train_model.best_model.predict(train_model.X_test)

plt.figure(figsize=(8, 8))

plt.scatter(
    train_model.y_test,
    predictions,
    alpha=0.5,
    color="#4C72B0",
)

min_value = min(train_model.y_test.min(), predictions.min())
max_value = max(train_model.y_test.max(), predictions.max())

plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    color="black",
    linewidth=1,
    linestyle="--",
)

plt.title(f"Actual vs Predicted Calories Burned ({train_model.best_model_name})")
plt.xlabel("Actual Calories Burned")
plt.ylabel("Predicted Calories Burned")

plt.show()
```

```
C:\Users\ayman\AppData\Local\Temp\ipykernel_9960\789238704.py:27: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()
```

Feature Importance / Coefficients (Best Model)

English

Tree-based models (Decision Tree , Random Forest) expose `feature_importances_`; Linear Regression exposes `coef_` instead. We show whichever applies to the saved best model.

العربية

بينما `feature_importances_` تُتيح (Decision Tree ، Random Forest) النماذج الشجرية ؛ نعرض ما يناسب أفضل نموذج محفوظ. `coef_` بدلاً منها Linear Regression يُتيح.

```
In [7]: if hasattr(train_model.best_model, "feature_importances_"):
        values = train_model.best_model.feature_importances_
        value_label = "Feature Importance"
    elif hasattr(train_model.best_model, "coef_"):
        values = train_model.best_model.coef_
        value_label = "Coefficient"
    else:
        values = None

    if values is not None:
        importance = (
            pd.Series(values, index=train_model.X.columns)
            .sort_values()
        )

        plt.figure(figsize=(9, 6))
        plt.barh(importance.index, importance.values, color="#55A868")
        plt.title(f"{value_label} - {train_model.best_model_name}")
        plt.xlabel(value_label)
        plt.ylabel("Feature")
        plt.axvline(0, color="black", linewidth=0.8)
        plt.show()
    else:
        print(f"{train_model.best_model_name} does not expose feature_importances_ or c
```

```
C:\Users\ayman\AppData\Local\Temp\ipykernel_9960\2739122779.py:22: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()
```

Summary

English

- **Evaluation:** `src/evaluate_model.py` compared Linear Regression, Decision Tree, and Random Forest with MAE, MSE, RMSE, and R^2 and identified the best model by R^2 .
- **Training & persistence:** `src/train_model.py` re-trained the same three models on the same split and saved the best one to `models/best_model.pkl`.
- **Verification:** the saved file was reloaded with `joblib` to confirm it matches the model type reported as best.
- **Diagnostics:** the actual-vs-predicted scatter and the feature-importance / coefficient chart are new additions here (not duplicated from any `src/` file) to visually inspect the saved model's behavior on the test set.
- **Note:** both scripts split and train independently with the same `random_state=42`, so results are reproducible but the training itself runs twice — once for the comparison table, once for the saved artifact.

العربية

- **التقييم:** `src/evaluate_model.py` بين Linear Regression وDecision Tree وRandom Forest بأفضل نموذج حسب R^2 ، وRMSE وMSE وMAE باستخدام `src/evaluate_model.py`.
- **التدريب والحفظ:** `src/train_model.py` أعادت تدريب نفس النماذج الثلاثة على نفس `models/best_model.pkl` التقسيم وحفظت الأفضل في `models/best_model.pkl`.
- **التأكد من تطابقه مع نوع النموذج:** `joblib` **التحقق:** تمت إعادة تحميل الملف المحفوظ باستخدام `joblib` للتحقق من تطابقه مع نوع النموذج.
- **التشخيص:** مخطط القيم الفعلية مقابل المتوقعة ومخطط أهمية الخصائص/المعاملات هما لفحص سلوك النموذج (`src/`) وليستا منسوختين من أي ملف في (إضافتان جديدتان هنا المحفوظ بصريًا على مجموعة الاختبار).
- **ملاحظة:** `random_state=42` ، لذلك النتائج ، كلا السكريبتين يقسمان ويدربان بشكل مستقل بنفس قابلية لإعادة الإنتاج، لكن التدريب نفسه يُنفَّذ مرتين — مرة لجدول المقارنة، ومرة للنموذج المحفوظ.